MACHINE LEARNING FUNDAMENTALS SERIES

# Reinforcement Learning with Neural Networks

Policy Gradients Method, Cross-Entropy Loss, Reward Feedback & Backpropagation Mathematics

## 1. Introduction and Problem Formulation

In standard supervised learning, neural networks are trained using backpropagation with known ground-truth targets ($y$). However, in many decision-making scenarios, the ideal target action is unknown in advance. Instead, an agent interacts with an environment, takes an action, and receives a scalar **Reward** signal based on the outcome.

To optimize neural networks under environmental feedback, we employ **Policy Gradients**—a foundational Reinforcement Learning (RL) algorithm.

**Benchmark Decision Scenario: Snack Selection**

Consider an agent deciding where to buy a French fry snack based on hunger level ($H \in [0.0, 1.0]$):

- **Option 1 (Squatch's Fry Shack):** Preferred when not hungry ($H = 0.0$) because it serves smaller portions.
- **Option 2 (Norm's Fry Hut):** Preferred when very hungry ($H = 1.0$) because it serves large portions.

Because portion sizes are initially unknown, standard backpropagation cannot be applied directly. The network must learn through stochastic action sampling, guessing, and reward feedback.

## 2. Network Architecture and Action Sampling Pipeline

The benchmark neural network takes a continuous hunger input $H$, applies weight $W$ and bias $B$, and routes the output through a Sigmoid activation function:

$$X = H \cdot W + B, \qquad P_{\text{Norm}} = \sigma(X) = \frac{1}{1 + e^{-X}}, \qquad P_{\text{Squatch}} = 1 - P_{\text{Norm}} \tag{1}$$

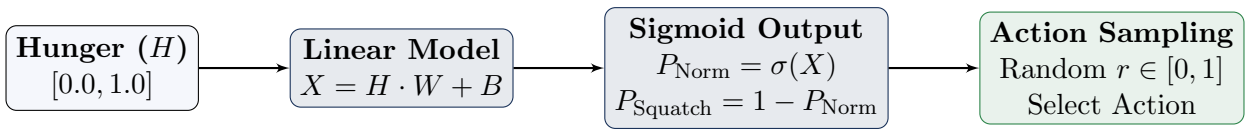



Figure 1: Policy Gradient Neural Network Architecture and Stochastic Action Sampling Pipeline.

### 2.1 Stochastic Action Selection

To select an action, the agent draws a random uniform number $r \sim \mathcal{U}(0, 1)$:

- If $0 \leq r \leq P_{\text{Squatch}}$, the agent visits **Squatch's Fry Shack**.
- If $P_{\text{Squatch}} < r \leq 1.0$, the agent visits **Norm's Fry Hut**.

## 3. Cross-Entropy Loss Derivation via Chain Rule

When the agent takes an action, it temporarily **guesses** that the chosen action was optimal ($P_{\text{ideal}} = 1.0$). To quantify the discrepancy between the network's output probability and the guessed ideal state, we compute **Cross-Entropy Loss**.

### 3.1 Cross-Entropy Formulations

- **If Action = Squatch:** $C_{\text{Squatch}} = -\ln(P_{\text{Squatch}}) = -\ln(1 - P_{\text{Norm}})$.
- **If Action = Norms:** $C_{\text{Norm}} = -\ln(P_{\text{Norm}})$.

### 3.2 Chain Rule Breakdown for Bias $B$

To optimize parameter $B$, we apply the Chain Rule to differentiate Cross-Entropy Loss $C$ with respect to bias $B$:

$$\frac{\partial C}{\partial B} = \frac{\partial C}{\partial P_{\text{Norm}}} \cdot \frac{\partial P_{\text{Norm}}}{\partial X} \cdot \frac{\partial X}{\partial B} \tag{2}$$

#### 3.2.1 Step 1: Derivative of Sigmoid Output wrt Linear Input ($\frac{\partial P_{\text{Norm}}}{\partial X}$)

Differentiating the Sigmoid function $P_{\text{Norm}} = (1 + e^{-X})^{-1}$ using the Chain and Power Rules:

$$\frac{\partial P_{\text{Norm}}}{\partial X} = -(1+e^{-X})^{-2} \cdot (-e^{-X}) = \frac{e^{-X}}{(1+e^{-X})^2} = \left(\frac{1}{1+e^{-X}}\right)\left(1 - \frac{1}{1+e^{-X}}\right) = \mathbf{P}_{\text{Norm}}(\mathbf{1} - \mathbf{P}_{\text{Norm}}) \tag{3}$$

#### 3.2.2 Step 2: Derivative of Linear Input wrt Bias ($\frac{\partial X}{\partial B}$)

Since $X = H \cdot W + B$, the partial derivative with respect to bias $B$ is:

$$\frac{\partial X}{\partial B} = \mathbf{1} \tag{4}$$

#### 3.2.3 Step 3: Differentiating Cross-Entropy for Specific Actions

**Case A: Action = Squatch's Fry Shack ($C_{\text{Squatch}} = -\ln(1 - P_{\text{Norm}})$)**

$$\frac{\partial C_{\text{Squatch}}}{\partial P_{\text{Norm}}} = -\frac{1}{1 - P_{\text{Norm}}} \cdot (-1) = \frac{\mathbf{1}}{\mathbf{1} - \mathbf{P}_{\text{Norm}}} \tag{5}$$

Multiplying the three chain rule components yields the uncorrected gradient:

$$\frac{\partial C_{\text{Squatch}}}{\partial B} = \left(\frac{1}{1 - P_{\text{Norm}}}\right) \cdot [P_{\text{Norm}}(1 - P_{\text{Norm}})] \cdot 1 = \mathbf{P}_{\text{Norm}} \tag{6}$$

*Numerical Evaluation:* At initial state $B = 0.0$ and $H = 0.0$, $P_{\text{Norm}} = 0.5$. The uncorrected slope is $+\mathbf{0.5}$.

**Case B: Action = Norm's Fry Hut ($C_{\text{Norm}} = -\ln(P_{\text{Norm}})$)**

$$\frac{\partial C_{\text{Norm}}}{\partial P_{\text{Norm}}} = -\frac{1}{P_{\text{Norm}}} \implies \frac{\partial C_{\text{Norm}}}{\partial B} = \left(-\frac{1}{P_{\text{Norm}}}\right) \cdot [P_{\text{Norm}}(1 - P_{\text{Norm}})] \cdot 1 = -(\mathbf{1} - \mathbf{P}_{\text{Norm}}) \tag{7}$$

*Numerical Evaluation:* At $P_{\text{Norm}} = 0.4$, the uncorrected slope is $-(1 - 0.4) = -\mathbf{0.6}$.

## 4. Reward Adjustment and Policy Gradient Weight Update

The uncorrected gradient $\frac{\partial C}{\partial B}$ is calculated under the assumption that the taken action was correct. However, if the environmental outcome reveals the action was wrong, applying $\frac{\partial C}{\partial B}$ directly would reinforce an incorrect behavior.

To fix this, Policy Gradients multiply the uncorrected gradient by a scalar **Reward** ($R$):

$$\text{Gradient}_{\text{adjusted}} = \frac{\partial C}{\partial B} \times R \tag{8}$$

**The Core Mathematical Mechanism of Policy Gradients**

- **Positive Reward ($R = +1.0$):** Correct guess (e.g., getting a small portion when not hungry). Multiplying by $+1.0$ preserves the gradient direction, causing Gradient Descent to increase the probability of taking that action again.
- **Negative Reward ($R = -1.0$):** Incorrect guess (e.g., getting a huge portion when not hungry). Multiplying by $-1.0$ **flips the direction of the gradient**, forcing Gradient Descent to update weights in the opposite direction and reduce the probability of taking that incorrect action.

### 4.1 Parameter Update Rule (Gradient Descent)

Using the adjusted gradient, parameter $B$ is updated via standard Gradient Descent with learning rate $\eta$:

$$B_{\text{new}} = B_{\text{old}} - \eta \cdot \text{Gradient}_{\text{adjusted}} = B_{\text{old}} - \eta \cdot \left( \frac{\partial C}{\partial B} \cdot R \right) \tag{9}$$

### 4.2 Detailed Numerical Training Execution

#### 4.2.1 Episode 1: Input $H = 0.0$ (Not Hungry), Initial $B^{(0)} = 0.0$

1. **Forward Pass:** $P_{\text{Norm}} = 0.5, P_{\text{Squatch}} = 0.5$.
2. **Action Selection:** Random sample $r = 0.2 \leq 0.5 \implies$ Visit **Squatch's Fry Shack**.
3. **Uncorrected Slope:** $\frac{\partial C_{\text{Squatch}}}{\partial B} = P_{\text{Norm}} = +0.5$.
4. **Environment Feedback:** Serves a small portion. Since $H = 0.0$, small portion is good $\implies$ $\mathbf{R = +1.0}$.
5. **Adjusted Gradient:** $+0.5 \times (+1.0) = \mathbf{+0.5}$.
6. **Parameter Update ($\eta = 1.0$):** $B^{(1)} = 0.0 - (1.0 \times 0.5) = \mathbf{-0.5}$.
7. **Effect:** $P_{\text{Squatch}}$ increases from 0.5 to $\sigma(-(-0.5)) = 0.6$ (higher probability of going to Squatch when not hungry).

#### 4.2.2 Episode 2: Input $H = 0.0$ (Not Hungry), Current $B^{(1)} = -0.5$

1. **Forward Pass:** $P_{\text{Norm}} = 0.4, P_{\text{Squatch}} = 0.6$.
2. **Action Selection:** Random sample $r = 0.9 > 0.6 \implies$ Visit **Norm's Fry Hut**.
3. **Uncorrected Slope:** $\frac{\partial C_{\text{Norm}}}{\partial B} = -(1 - 0.4) = -0.6$.
4. **Environment Feedback:** Serves a huge portion. Since $H = 0.0$, huge portion is bad $\implies$ $\mathbf{R = -1.0}$.
5. **Adjusted Gradient (Error Correction):** $-0.6 \times (-1.0) = \mathbf{+0.6}$ (Gradient direction is flipped!).
6. **Parameter Update ($\eta = 1.0$):** $B^{(2)} = -0.5 - (1.0 \times 0.6) = \mathbf{-1.1}$.
7. **Effect:** $P_{\text{Norm}}$ drops further to $\sigma(-1.1) \approx 0.25$, reinforcing preference for Squatch when $H = 0$.

## 5. Summary Comparison: Policy Gradient vs. Supervised Backpropagation

Table 1: Methodological Comparison of Supervised vs. Reinforcement Learning Weight Updates

| Aspect / Property | Supervised Backpropagation | Policy Gradient Reinforcement Learning |
|---|---|---|
| **Target Signal** | Explicit Ground Truth ($y$) | Scalar Environmental Reward ($R$) |
| **Loss Function** | Direct MSE or Cross-Entropy | Action-based Cross-Entropy |
| **Gradient Direction** | Always points toward target $y$ | Initial gradient assumes guess is correct |
| **Gradient Correction** | Fixed by loss function | **Flipped/scaled by Reward $R$** |
| **Action Selection** | Deterministic argmax/output | Stochastic sampling ($r \sim \mathcal{U}(0,1)$) |
| **Convergence Goal** | Minimize prediction residual | Maximize expected cumulative reward ($B \to -10.0$) |